Configuration

Pre- Requisite Commands and Arguments in Docker

UC1: you are running Ubuntu container

```
docker run ubuntu
docker ps
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS

docker ps -a
CONTAINER ID    IMAGE    COMMAND       CREATED          STATUS                      PORTS
45aacca36850    ubuntu   "/bin/bash"   43 seconds ago   Exited (0) 41 seconds ago
```

Unlike a VM containers are not meant to host an O/S, containers are meant to host specific task or process, such as to host an Instance of the webserver, an instance of an application server or db server instance or to carry out some computation.

Once the task is over container exits. Container lives only till process inside it alive, if the webservice inside the container stops or crashes the container exits.

Who defines what process runs inside the container?

CMD instruction inside Dockerfile that defines the program that will be run inside the container when it will be start.

CMD [“nginx”] // docker image of nginx

CMD [“mysqld”] // docker image of mysql

CMD [“bash”] // docker image of ubuntu

Ubuntu Image uses a bash as the default command, but bash is not really a process like a webserver or the database server, it’s a shell that listens for input from a terminal , if it can not find the terminal it exits. When we ran the ubuntu container **docker run ubuntu** docker creates a container from ubuntu image and launch the bash program, by default docker doesn’t attach a terminal to a container when its is run, so that bash program doesn’t find a

terminal so that it exits. Since the process that was started when the container was created finished the container exits as well.

How to specify a different command to start the container?

Option1: append the docker run command

It will overwrite the default command specified with in the image.

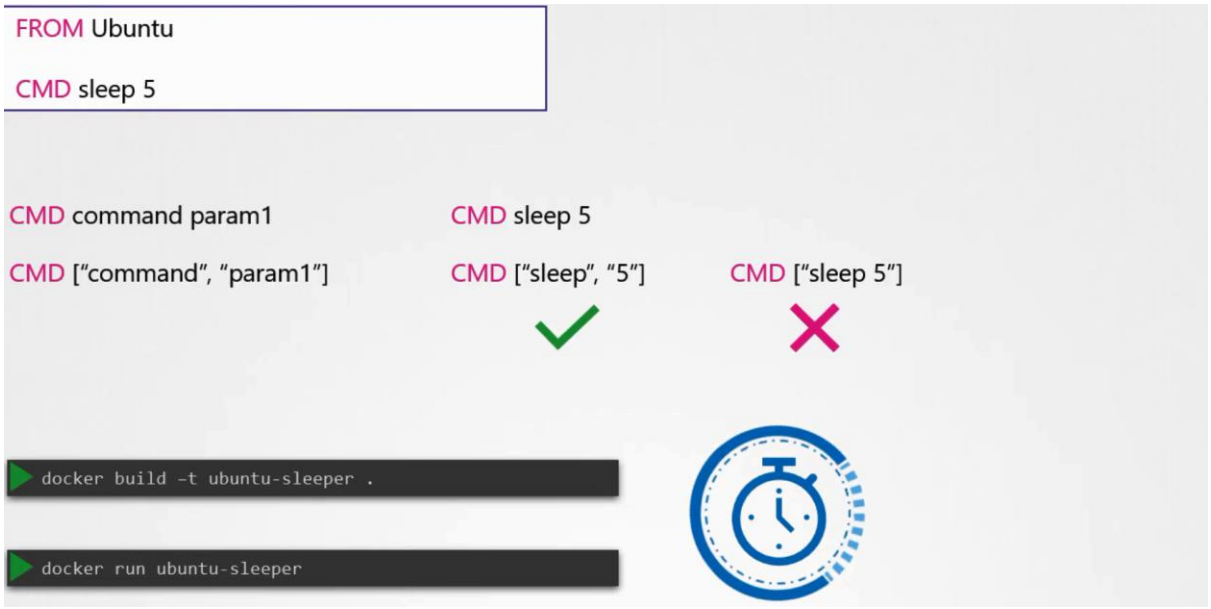
```
docker run ubuntu [COMMAND]

docker run ubuntu sleep 5
```

In this case when container will start, it will execute the sleep program and then exits.

UC: How to make this change permanent say you want image always run the sleep program when it starts?

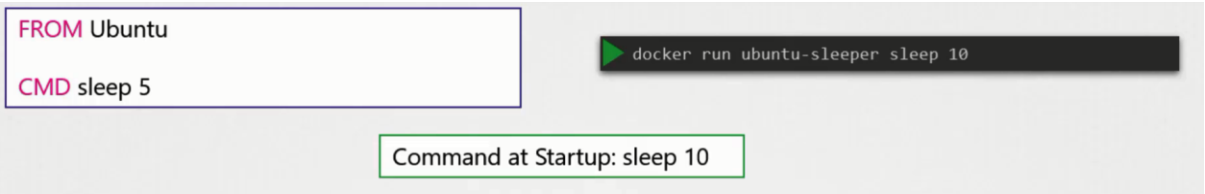
You have to create your own image from the base ubuntu image and specify your own **CMD**



There are different ways of specify the same, if you specify the same in JSON array format then first element of the array will be always be an executable. **Don't** specify the command and parameter together. Now can build new image using the **docker build -t ubuntu-sleeper.**

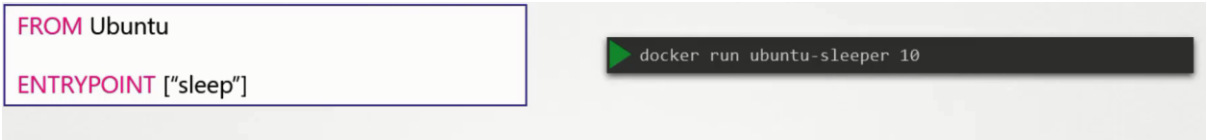
docker run ubuntu-sleeper //It will always sleep for 5 sec and then exits.

UC: Want to change the number of seconds it sleeps currently its hard coded for 5 sec?



As we learned before one option is to run the docker run command with the new **command appended to it**. In this case **sleep 10, and** so the command that will be run at startup will be sleep 10. But it doesn't look very good. The name of the image **ubuntu-sleeper** in itself implies that the container will sleep. So we shouldn't have to specify the sleep command again. We want to pass only the number of seconds, and sleep command must be invoked automatically

docker run ubuntu-sleeper 10



ENTRYPOINT instruction is like a command instruction, you can specify the program that will be invoked when the container starts. Whatever you will specify in ENTRYPOINT will automatically appended when you run the container.

In case of CMD instruction the **command line** parameter passed will gets replaced entirely, while in case of ENTRYPOINT command line parameters gets appended.

UC: What will happen if in second case will run the command without append the number of seconds?

Then the command at the start-up will be just **sleep** and we get the exception "missing operand", so you have to add default value as well.

UC: How to define the default value for the command line instrunction?

You have to define ENTRYPOINT and CMD in JSON format

FROM ubuntu

ENTERYPOINT [“sleep”]

CMD [“5”]

So at startup, the **command would be sleep 5**, if you didn’t specify any parameters in the command line. **If you did, then that will override the command instruction,** and remember, for this to happen, you should always specify the entry point and command instructions in a JSON format.

UC: How to overwrite the ENTRYPOINT instruction?

You can override it by using the **--entrypoint** option in the docker run command. The final command at startup would then be

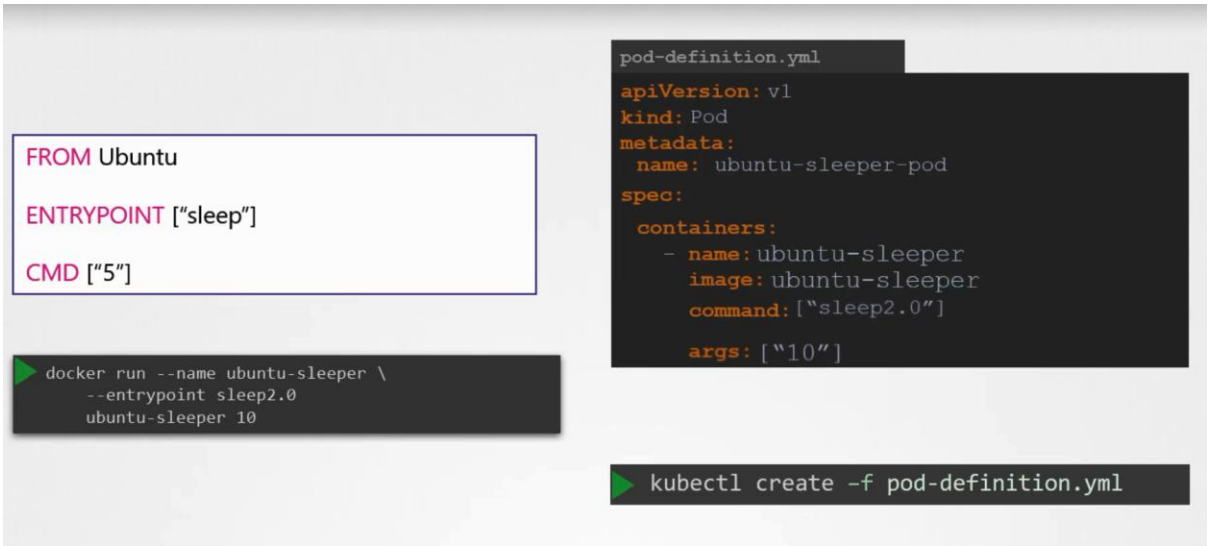
docker run --entrypoint sleep2.0 ubuntu-sleeper

docker run --entrypoint sleep2.0 ubuntu-sleeper 10

Commands and Arguments In K8s:

In the previous lecture we created a simple docker image that sleeps for a given number of seconds. We named it ubuntu-sleeper and we ran it using the docker command **docker run ubuntu-sleeper**. By default it sleeps for 5 seconds, but you can override it by passing a command line argument.

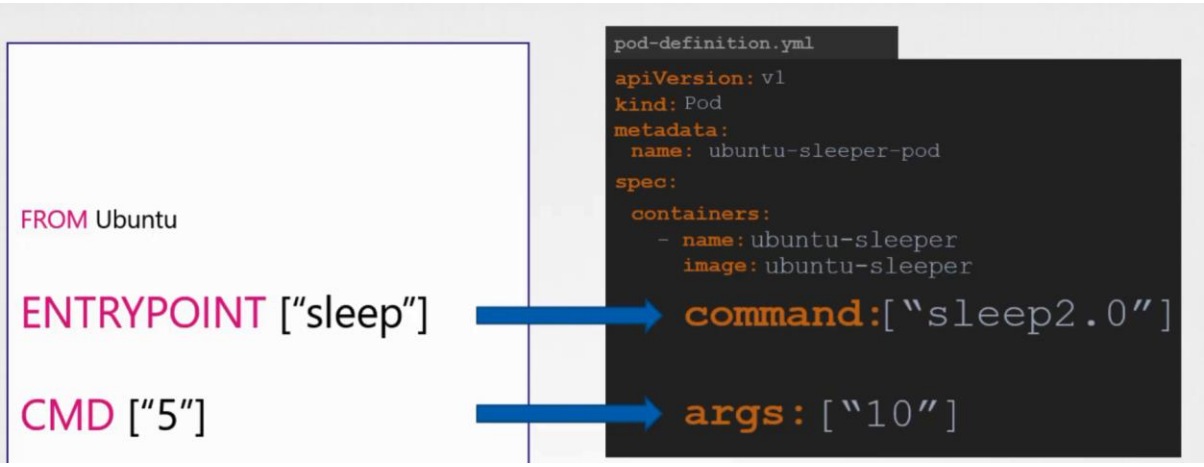
We will now create a pod using this image. We start with a blank pod definition template, input the name of the pod and specify the image name. When the pod is created, it creates a container from the specified image, and the container sleeps for 5 seconds before exiting. Now, if you need the container to sleep for 10 seconds as in the second command, how do you specify the additional argument in the pod-definition file? Anything that is appended to the docker run command will go into the “**args**” property of the pod definition file, in the form of an array like this.



Let us try to relate that to the Dockerfile we created earlier. **The Dockerfile has an Entrypoint as well as a CMD instruction specified. The entrypoint is the command that is run at startup, and the CMD is the default parameter passed to the command.**

With the **args** option in the pod-definition file we override the CMD instruction in the Dockerfile. But what if you need to override the entrypoint?

The corresponding entry in the pod definition file would be using a **command** option.



So to summarize, there are two fields that correspond to two instructions in the Dockerfile. The **command overrides the entrypoint** instruction and the **args field overrides the CMD** instruction in the Dockerfile. Remember the command field does not override the CMD instruction in the Dockerfile.

Use environment variables to define arguments

In the preceding example, you defined the arguments directly by providing strings. As an alternative to providing strings directly, you can define arguments by using environment variables:

Run a command in a shell

In some cases, you need your command to run in a shell. For example, your command might consist of several commands piped together, or it might be a shell script. To run your command in a shell, wrap it like this:

command: ["/bin/sh"]

args: ["-c", "while true; do echo hello; sleep 10; done"]

Notes:

This table summarizes the field names used by Docker and Kubernetes.

Description	Docker field name	Kubernetes field name
The command run be container	ENTRYPOINT	command
The arguments passed to the command	CMD	args

When you override the default ENTRYPOINT and CMD, these rules apply:

- If you do not supply command or args for a Container, the defaults defined in the Docker image are used.
- If you supply a command but no args for a Container, only the supplied command is used. The default ENTRYPOINT and the default CMD defined in the Docker image are ignored.
- If you supply only args for a Container, the default Entrypoint defined in the Docker image is run with the args that you supplied.
- If you supply a command and args, the default Entrypoint and the default CMD defined in the Docker image are ignored. Your command is run with your args.

Here are some examples:

Image Entrypoint	Image CMD	Container command	Container args	Command run
[/ep-1]	[foo bar]	<not set>	<not set>	[ep-1 foo bar]
[/ep-1]	[foo bar]	[/ep-2]	<not set>	[/ep-2]
[/ep-1]	[foo bar]	<not set>	[zoo boo]	[ep-1 zoo boo]
[/ep-1]	[foo bar]	[/ep-2]	[zoo boo]	[ep-2 zoo boo]